

IntuiUX Agent

Описание кода для разработчиков

Версия	1.1.0
Фреймворк	Next.js 16 (App Router)
Язык	TypeScript 5
Стилизация	Tailwind CSS 4 + shadcn/ui
База данных	Prisma ORM + SQLite
AI SDK	z-ai-web-dev-sdk
Диаграммы	Mermaid.js 11
Деплой	Vercel
Репозиторий	Wasp1977/intuiux_092026
Лицензия	MIT

1. Обзор архитектуры

IntuiUX Agent представляет собой одностраничное веб-приложение на базе Next.js 16 с App Router, реализующее 9-этапный UX-пайплайн для автоматизированного создания пользовательских интерфейсов. Система принимает на вход текстовое описание продукта или транскрипцию бизнес-встречи, последовательно обрабатывает данные через 9 этапов пайплайна, и на выходе генерирует интерактивный HTML-прототип, опубликованный на GitHub, а также сопутствующие артефакты: конкурентный анализ, CJM, информационную архитектуру, пользовательские сценарии, приглашения на тестирование, гайдлайны и метрики.

Архитектура следует паттерну Server-First API: вся бизнес-логика и вызовы LLM выполняются на сервере через API-маршруты Next.js, а клиент отвечает только за отображение UI и управление состоянием пайплайна. Связь между клиентом и сервером осуществляется через REST API вызовы из компонента page.tsx к маршрутам /api/pipeline/stages/*. Данные пайплайна сохраняются в SQLite через Prisma ORM.

1.1. Ключевые принципы

- Модульность: каждый этап пайплайна — независимый API-маршрут в /api/pipeline/stages/
- Контекстность: каждый этап получает выходные данные предыдущих этапов через Prisma
- LLM-управляемость: основная генерация контента выполняется через z-ai-web-dev-sdk
- Минимальный клиент: page.tsx только отображает артефакты и управляет навигацией
- Изолированный рендеринг: прототипы HTML рендерятся в sandboxed iframe
- Mermaid-визуализация: CJM, IA и Userflow отображаются как SVG-диаграммы

1.2. Стек технологий

Слой	Технология	Назначение
Frontend	React 19 + Next.js 16	UI, маршрутизация, SSR
UI Kit	shadcn/ui (Radix)	40+ компонентов интерфейса
Стилизация	Tailwind CSS 4	Utility-first CSS
Backend	Next.js API Routes	REST API, LLM-вызовы
AI/LLM	z-ai-web-dev-sdk	Генерация контента
Диаграммы	Mermaid.js 11	CJM, IA, Userflow SVG
БД	Prisma ORM + SQLite	Хранение сессий и этапов
Деплой	Vercel	Production хостинг

2. Структура проекта

Проект следует стандартной структуре Next.js App Router. Исходный код разделён на компоненты (src/components/), библиотечные модули (src/lib/), API-маршруты (src/app/api/) и хуки (src/hooks/). Каждый этап пайплайна имеет собственный API-маршрут в директории src/app/api/pipeline/stages//route.ts, что обеспечивает модульность и независимость этапов.

2.1. Корневые конфигурационные файлы

Файл	Назначение
package.json	Зависимости и скрипты (dev, build, start, db:*)
next.config.ts	Конфигурация Next.js (standalone output)
tailwind.config.ts	Настройка Tailwind CSS (тёмная тема)
tsconfig.json	TypeScript конфигурация (strict, paths: @/*)
prisma/schema.prisma	Схема БД (8 моделей)
vercel.json	Конфигурация Vercel деплоя
components.json	Конфигурация shadcn/ui
eslint.config.mjs	ESLint правила

2.2. Директория src/

Путь	Назначение
src/app/page.tsx	Главная страница — UI пайплайна (~2000 строк)
src/app/layout.tsx	Root layout — шрифты, метаданные, темы
src/app/globals.css	Глобальные стили (Tailwind, тёмная тема)
src/components/MessageContent.tsx	Рендеринг артефактов (MD, Mermaid, iframe)
src/components/MermaidRenderer.tsx	Mermaid диаграммы (SVG рендеринг)
src/components/ui/*	40+ shadcn/ui компонентов
src/lib/zai.ts	LLM класс — обёртка над z-ai-web-dev-sdk
src/lib/db.ts	Prisma Client singleton
src/lib/pipeline-config.ts	Конфигурация этапов пайплайна
src/lib/idea-extractor.ts	Алгоритмическое извлечение идеи
src/lib/competitor-analyzer.ts	Анализ конкурентов с SWOT

Путь	Назначение
src/lib/context-aware-competitor-search.ts	Контекстный поиск конкурентов
src/lib/cjm-generator.ts	Генерация Customer Journey Map
src/lib/prototype-generator.ts	Генерация HTML-прототипов
src/lib/agents-fallback.ts	Fallback данные для LLM-агентов
src/lib/agent-context.ts	Контекст для LLM-агентов
src/lib/github-pages.ts	Публикация на GitHub Pages
src/lib/vercel-deploy.ts	Деплой прототипов на Vercel
src/lib/utils.ts	Утилиты (cn()) для className merge)
src/hooks/use-toast.ts	Toast notifications hook
src/hooks/use-mobile.ts	Mobile detection hook

3. API маршруты

API-маршруты составляют основу серверной логики. Каждый этап пайплайна реализован как независимый POST-эндпоинт, принимающий sessionId и опциональную корректировку. Маршрут обновляет статус этапа в БД (pending -> processing -> completed/error), вызывает LLM с контекстно-зависимым промптом, парсит ответ, генерирует Mermaid-диаграмму (если требуется), и сохраняет результат. Это обеспечивает трассируемость, возобновляемость и аналитику каждого запуска.

3.1. Главный маршрут: /api/chat/route.ts

Основной API-маршрут для большинства этапов пайплайна. Принимает POST с полями message, stage, history. Определяет тип агента через AGENT_PROMPTS на основе текущего этапа. Формирует system prompt и user prompt с контекстом предыдущих этапов. Вызывает LLM через z-ai-web-dev-sdk. При ошибке использует getFallbackResponse() для генерации базового контента. Возвращает результат клиенту как текстовый ответ.

Ключевой механизм: AGENT_PROMPTS — словарь системных промптов для каждого этапа, определяющий роль LLM и требования к выходному формату. Промпты составлены на русском языке и содержат инструкции по структуре артефакта. Функция getFallbackResponse() обеспечивает работу системы при недоступности LLM.

3.2. Маршрут пайплайна: /api/pipeline/route.ts

Управляет сессиями пайплайна через POST с параметром action. Поддерживаемые действия: create_session (с автоопределением типа продукта через detectProductType), get_session,

get_all_sessions, delete_session, approve_stage, update_stage_status, stop_generation, detect_product_type. Сессия содержит массив этапов PipelineStage со статусами, входными/выходными данными, Mermaid-кодом и ошибками.

3.3. Маршруты этапов: /api/pipeline/stages/*

Этап	Маршрут	Вход	Выход	LLM
Idea	/stages/idea/	inputText	title, problem, solution, keyFeatures	Да
Competitors	/stages/competitors/	sessionId	direct/indirect competitors, SWOT	Да+Поиск
CJM	/stages/cjm/	ideaData	persona, journeyStages, Mermaid	Да
IA	/stages/ia/	ideaData, cjmData	pages, navigation, Mermaid	Да
Userflow	/stages/userflow/	ideaData, cjmData, iaData	scenarios, interactions, Mermaid	Да
Prototype	/stages/prototype/	ideaData, iaData, userflowData	pages[].html	Да
Invitation	/stages/invitation/	ideaData, prototypeData	email, telegram, screener	Да
Guidelines	/stages/guidelines/	ideaData, prototypeData	tasks, questions, guide	Да
Metrics	/stages/metrics/	ideaData, cjmData	northStar, AARRR, recs	Да

3.4. Дополнительные API-маршруты

Маршрут	Назначение
/api/competitors-search/	Прямой поиск конкурентов через веб
/api/prototype/	Управление прототипами (CRUD)
/api/prototype/[slug]/	Доступ к прототипу по slug
/api/github/	Взаимодействие с GitHub API
/api/vercel-deploy/	Деплой прототипов на Vercel
/api/knowledge/	Управление базой знаний
/api/agents/	Управление AI-агентами
/api/insights/	Инсайты и рекомендации
/api/metrics/	Продуктовые метрики
/api/deploy/	Общий деплой-маршрут
/api/update-prototype-files/	Обновление файлов прототипов

4. Модуль LLM (zai.ts)

Модуль `src/lib/zai.ts` предоставляет абстракцию над `z-ai-web-dev-sdk` для вызовов LLM. Реализованы класс `LLM` и функция `createChatCompletion()`. Оба подхода читают конфигурацию из файла `.z-ai-config`, который ищется по порядку: текущая директория, домашняя директория пользователя, `/etc/`, `/tmp/`. Конфигурация содержит `baseUrl` и `apiKey`.

4.1. Класс LLM

Класс `LLM` реализует паттерн `lazy initialization`: конфигурация загружается при первом вызове `chat()` и кэшируется. Метод `chat()` принимает `ChatCompletionOptions` (`messages`, `temperature`, `maxTokens`, `thinking`), выполняет POST-запрос к `{baseUrl}/chat/completions` с заголовком `X-Token`, возвращает текстовый контент ответа. Температура по умолчанию `0.7`, `max_tokens` `4096`, `thinking` `disabled`.

4.2. Параметры вызова

Параметр	Тип	По умолч.	Описание
<code>messages</code>	<code>ChatMessage[]</code>	—	Массив сообщений (<code>system/user/assistant</code>)
<code>temperature</code>	<code>number</code>	<code>0.7</code>	Температура генерации (0-1)
<code>maxTokens</code>	<code>number</code>	<code>4096</code>	Макс. токенов в ответе
<code>thinking</code>	<code>{type: string}</code>	<code>disabled</code>	Режим расш. мышления

5. Конфигурация пайплайна (pipeline-config.ts)

Модуль определяет структуру пайплайна в зависимости от типа продукта. Поддерживается 8 типов: `ecommerce`, `saas`, `b2b`, `blog`, `landing`, `dashboard`, `booking`, `app`. Для каждого типа определён набор этапов с зависимостями и флагами опциональности. Функция `detectProductType()` использует систему весов на основе регулярных выражений для автоопределения типа продукта из текста.

5.1. Определение типа продукта

`detectProductType(text)` анализирует текст с помощью набора регулярных выражений для каждого типа. Каждое совпадение добавляет вес. Например, "магазин", "товар" добавляет +3 к `ecommerce`, "запись", "бронирование" — +3 к `booking`. Тип с максимальным весом выбирается как результат. По умолчанию — `"landing"`.

5.2. Вариации этапов

Тип	Обязательные	Оptionальные
ecommerce	Idea, Competitors, CJM, IA, Userflow, Prototype	Invitation, Guidelines, Metrics
saas	Idea, Competitors, CJM, IA, Userflow, Prototype	Invitation, Guidelines, Metrics
b2b	Idea, Competitors, CJM, IA, Userflow, Prototype	Invitation, Guidelines, Metrics
blog	Idea, Competitors, CJM, IA, Userflow, Prototype	Metrics
landing	Idea, Competitors, CJM, IA, Prototype	Metrics
dashboard	Idea, Competitors, CJM, IA, Userflow, Prototype	Metrics
booking	Idea, Competitors, CJM, IA, Userflow, Prototype	Invitation, Guidelines, Metrics
app	Idea, Competitors, CJM, IA, Userflow, Prototype	Invitation, Guidelines, Metrics

6. База данных (Prisma + SQLite)

Схема в `prisma/schema.prisma` содержит 8 моделей. Prisma Client инициализируется в `src/lib/db.ts` как singleton с логированием запросов и защитой от дублирования при hot-reload.

6.1. Модели данных

Модель	Назначение	Ключевые поля
Agent	AI-агенты	name, type, systemPrompt, isActive
KnowledgeBase	База знаний	title, content, source, category, tags
Conversation	Диалоги	agentId, title, summary
Message	Сообщения	conversationId, role, content
Insight	Инсайты	type, title, priority, status
Metric	Метрики	name, value, unit, category
UserSession	Исслед. сессии	type, transcript, painPoints
PrototypeDeployment	Деплой прототипов	slug, htmlContent, vercelUrl
PipelineSession	Сессии пайплайна	name, productType, status
PipelineStage	Этапы пайплайна	stageName, status, outputData, mermaidCode

6.2. Ключевые отношения

PipelineSession имеет отношение один-ко-многим с PipelineStage (каскадное удаление). PipelineStage имеет уникальный композитный индекс [sessionId, stageNumber]. Agent связан с Conversation, KnowledgeBase и Insight. Все модели используют CUID-идентификаторы и

автоматические timestamp-поля.

7. Клиентская часть (page.tsx)

Компонент `src/app/page.tsx` — основная страница приложения ("use client"). Управляет полным жизненным циклом пайплайна: от ввода текста до отображения артефактов. Размер ~2000 строк кода, что объясняется комплексностью UI с 9 этапами, системой зума, обработкой файлов и демо-режимом.

7.1. Ключевые состояния

Состояние	Тип	Назначение
stages	StageConfig[]	Массив этапов пайплайна
currentStageIndex	number	Индекс текущего этапа
stageOutputs	Record	Артефакты каждого этапа
isProcessing	boolean	Флаг обработки
inputText	string	Входной текст
sessionId	string	ID сессии пайплайна
productType	string	Тип продукта
zoomLevel	number	Зум артефакта (50-200%)
progress	number	Прогресс (0-100%)

7.2. Процесс обработки этапа

`processStage()` — основная логика. Для Competitors — прямой вызов `/api/pipeline/stages/competitors`, для остальных — `/api/chat` с параметром `stage`. Процесс: отображение "processing", вызов API, обновление `stageOutputs` и прогресса, авто-переход к следующему этапу. При ошибке — сообщение с возможностью повторить. Поддерживается демо-режим для тестирования без LLM.

8. Рендеринг артефактов (MessageContent.tsx)

Компонент `MessageContent` определяет тип контента и выбирает метод отображения: HTML-прототипы (`sandboxed iframe`), Mermaid-диаграммы (SVG через `Mermaid.js`), Markdown (`react-markdown`). Поддерживает масштабирование через `zoomLevel` и бейдж веб-поиска для Competitors.

8.1. Mermaid-рендеринг

MermaidDiagram инициализирует Mermaid.js с тёмной темой и кастомными цветами. Рендеринг асинхронный: mermaid.render() генерирует SVG, вставляемый через dangerouslySetInnerHTML. Диаграмма оборачивается в контейнер с градиентным фоном, цвет зависит от этапа. При ошибке отображается исходный код.

8.2. Prototypelframe

HTML-прототипы рендерятся в sandboxed iframe (allow-scripts allow-same-origin). Авто-высота через чтение scrollHeight из contentDocument, ограничение 300-2000px. Обеспечивает корректное отображение без внутренней прокрутки.

9. Этапы пайплайна: детали

9.1. Idea

Извлекает структурированную идею из текста. Промпт: "Senior Product Analyst". JSON-ответ: title, problem, solution, targetAudience, keyFeatures[], valueProposition, searchKeywords[], roles[], painPoints[], useCases[], risks[], constraints[], industry. После парсинга вызывается detectProductType(), сессия обновляется с productType.

9.2. Competitors

Контекстный поиск v3.0: (1) extractIdeaFromText() из исходного текста, (2) searchCompetitorsWithContext(maxCompetitors=10, enrichWithLLM=true), (3) форматирование в HTML/Markdown, (4) SWOT-анализ каждого конкурента через LLM, (5) группировка по категориям (direct, indirect, incumbent).

9.3. CJM

Создаёт 2-3 персоны, Mermaid journey с 6-8 стадиями (оценки 1-5), таблицу стадий с touchpoints/эмоциями/pain points, критические точки оттока, инсайты. generateCJMMermaid() преобразует в Mermaid journey синтаксис (pain=-2, opportunity=4).

9.4. IA

Создаёт модель навигации (Mermaid flowchart TD), описание страниц с URL/ролями, таксономию сущностей (erDiagram), навигационную модель. generateEnhancedMermaid() — цвета по типу продукта, стили для root/section/page/entity.

9.5. Userflow

Happy path (flowchart с decision points), 3 альтернативных сценария, 4 edge cases, таблицы экранов (8+) и переходов. `generateUserflowMermaid()` — subgraph для каждого сценария, соединения между шагами и экранами.

9.6. Prototype

Генерирует интерактивный HTML. Дизайн-система: фон #0f1117, карточки #1a1d27, акцент #FACC15. SPA с `showScreen()`, адаптив, CSS transitions. Экраны: Splash, Onboarding, Dashboard, Магазин виджетов, Настройка, Уведомления, Профиль. LLM возвращает HTML в code block.

9.7. Invitation, Guidelines, Metrics

Invitation: email, Telegram, пост для соцсетей, скрининг, форма записи. Guidelines: подготовка, структура сессии, скрипт модератора, задачи с критериями, пост-тест SUS. Metrics: North Star, AARRR-воронка, специфичные метрики, дашборд, рекомендации.

10. Деплой и публикация

Развёрнуто на Vercel (<https://intuiux092026.vercel.app/>), репозиторий Wasp1977/intuiux_092026. Прототипы публикуются в `prototypes/{name}-{timestamp}/`. Модули `github-pages.ts` и `vercel-deploy.ts` обеспечивают публикацию через GitHub API и Vercel API соответственно.

Vercel: регион iad1, макс. длительность функций 300s (для LLM), CORS для API. Сборка standalone output. Требуются DATABASE_URL и .z-ai-config.